# Chat With Legal Application

**Course Name: GenAI**

**Medicaps University – Datagami Skill Based Course**

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1. | VISHAL RATHOD | EN22CS306059 |
| 2. | PALAK GUPTA | EN22CS306036 |
| 3. | JAY PRAKASH SINGH | EN22CS304031 |
| 4. | RAJVEER MUKATI | EN22CS304051 |
| 5. | KOMAL JADHAV | EN22CS303028 |
| 6. | DIYA GOYAL | EN22CS304025 |

**Group Name: G11D11**

**Project Number: DO-37**

**Industry Mentor Name: Mr. Yash Sir**

**University Mentor Name: Dr. Hemlata Patel**

**Academic Year: 2025-26**

Table of Contents

1. Introduction.

1.1. Scope of the document.

1.2. Intended Audience

1.3. System overview.

2. System Design.

2.1. Application Design

2.2. Process Flow.

2.3. Information Flow.

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue.

3. Data Design.

3.1. Data Model

3.2. Data Access Mechanism

3.3. Data Retention Policies

3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements

7.1. Security Aspects

7.2. Performance Aspects

**Future Scope & Enhancements**

# 1. Introduction

The **Chat with Legal Documents** system is a Generative AI based application that allows users to upload legal documents and interact with them using natural language queries. The solution uses a **Retrieval-Augmented Generation (RAG)** architecture, where the system first retrieves the most relevant portions of the uploaded documents and then uses a Large Language Model to generate a grounded response. This improves answer relevance, reduces hallucination, and keeps responses tied to document content. This overall system direction is consistent with the LLD's system overview and layered design.

## 1.1 Scope of the document

This HLD covers the overall architecture and major design decisions of the system, including:

- system architecture and modules
- end-to-end document upload and query flow
- high-level data design
- interfaces between modules
- session and state handling
- caching approach
- non-functional requirements such as security and performance

It focuses on **what the system does and how the main components interact**, rather than low-level implementation details. This scope is the higher-level counterpart of the LLD scope, which focuses on internal structure, modules, workflows, APIs, and information flow.

## 1.2 Intended Audience

This document is intended for:

- project evaluators and faculty
- solution architects and system designers
- developers and backend/frontend engineers
- testers and QA team
- deployment and operations teams

The audience mapping is aligned with the LLD's intended audience section.

### 1.3 System overview

The system enables a user to:

1. upload a legal document such as a contract, agreement, or report
2. extract and preprocess text from that document
3. split the text into chunks
4. convert those chunks into embeddings
5. store embeddings in a vector database
6. ask questions in natural language
7. retrieve relevant chunks from the vector store
8. generate a contextual answer through an LLM
9. display the answer along with supporting context/citations

The major subsystems identified in the LLD are frontend, backend APIs, document processing pipeline, vector database, and LLM API integration.

---

## 2. System Design

The application follows a **modular layered architecture**. At a high level, the user interacts with the frontend, which communicates with backend APIs. The backend orchestrates document ingestion, text extraction, chunking, embedding creation, vector storage, retrieval, and answer generation. This matches the architecture shown in the LLD and its architecture diagram.

### 2.1 Application Design

The application is divided into the following high-level layers:

#### 1. Presentation Layer

- Web-based user interface
- Supports login, document upload, document list, and chat interaction
- Displays answers and source references

### 2. Application Layer

- Exposes REST APIs for frontend interaction
- Manages authentication, document lifecycle, chat requests, and orchestration

### 3. Processing Layer

- Handles parsing, cleaning, chunking, and embedding generation
- Converts unstructured documents into searchable semantic data

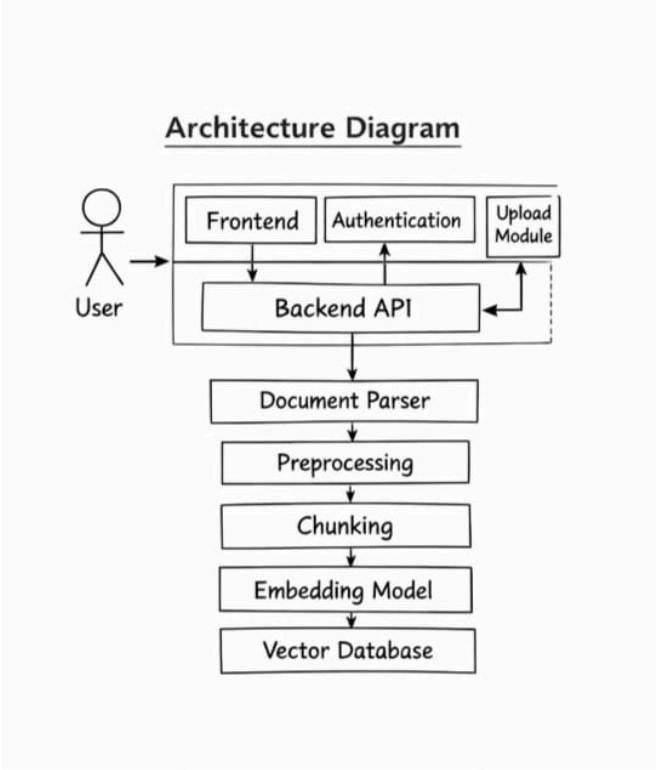
### 4. Data Layer

- Stores metadata in a relational database
- Stores embeddings in a vector database
- Maintains chat records and document status

### 5. External Service Layer

- Connects to embedding model service
- Connects to the LLM for answer generation

This follows the same layered structure already identified in the LLD: presentation, application, processing, data, and external services.




## 2.2 Process Flow

### A. Document Upload Flow

1. User logs into the system

2. User uploads a legal document
3. Backend validates file type and size
4. Document processor extracts raw text
5. Text is cleaned and normalized
6. Chunking module splits text into meaningful overlapping chunks
7. Embedding service converts chunks into vectors
8. Vectors are stored in vector database
9. Document metadata and processing status are saved
10. Document becomes available for question-answering

This matches the LLD upload process flow: upload, validate, extract, preprocess, chunk, embed, store, and mark ready.

### B. Query Processing Flow

1. User selects a document or document set

2. User submits a question in natural language
3. Query is converted into an embedding
4. Vector database performs similarity search
5. Top relevant chunks are retrieved

6. Query and retrieved context are passed to the LLM
7. LLM generates a grounded response
8. Backend returns the answer to frontend
9. Chat history is stored for future reference

This directly aligns with the LLD query flow.

## 2.3 Information Flow

The information flow is divided into two major phases.

**Upload Phase**

User → Frontend → Backend API → Document Processor → Chunking Module → Embedding Service → Vector Database → Metadata Store

**Query Phase**

User → Frontend → Backend API → Query Embedding → Vector Database → Retriever → LLM Service → Backend API → Frontend

The main data elements flowing through the system are document text, chunks, embeddings, user queries, retrieved context, and generated answers, exactly as stated in the LLD.

## 2.4 Components Design

**1. Frontend**

- Provides upload page, chat page, and dashboard
- Lets users manage documents and ask questions
- Displays answers with context references

**2. API Gateway / Backend API**

- Central orchestration layer
- Handles requests from frontend
- Applies authentication and authorization
- Routes upload and query requests to relevant services

**3. Authentication Module**

- Manages user registration/login
- Issues session or token-based authentication
- Restricts access to user-owned documents

**4. Document Processor**

- Parses PDF or supported document format
- Extracts readable text
- Handles formatting cleanup

**5. Preprocessing and Chunking Module**

- Cleans special characters and noise
- Splits text into manageable chunks
- Uses overlap to preserve context across chunks

**6. Embedding Service**

- Converts chunks and queries into vector embeddings
- Uses an embedding model compatible with semantic retrieval

**7. Vector Database**

- Stores chunk embeddings
- Supports similarity search and fast retrieval

**8. Metadata Database**

- Stores user details, document metadata, file names, processing status, timestamps, and chat references

**9. Retriever**

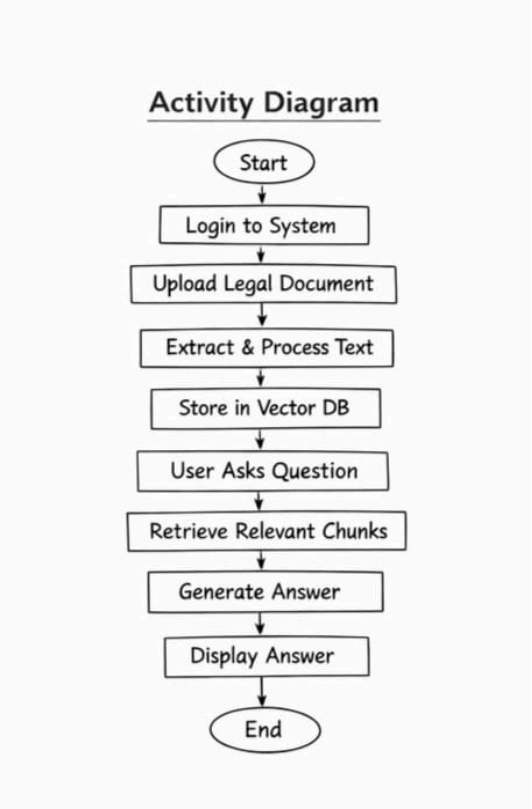
- Fetches top-k relevant chunks based on vector similarity
- Filters results by selected document and user access

**10. LLM Integration Module**

- Prepares prompt with query and retrieved context
- Calls the LLM API
- Returns grounded answer

**11. Chat History Module**

- Stores questions, answers, timestamps, and linked document IDs
- Enables session continuity and auditability




These components are a high-level representation of the modules already listed in the LLD, including frontend, backend API, document processor, chunking, embedding service, vector database, query processor, retriever, LLM integration, and chat history.

## 2.5 Key Design Considerations

The HLD is driven by the same design goals highlighted in the LLD.

**Accuracy**

- RAG ensures answers are based on uploaded documents

- Retrieved context is passed to the LLM to reduce hallucination

**Scalability**

- Document ingestion and query handling are modular
- Embedding generation, retrieval, and inference can be scaled independently

**Performance**

- Precomputed embeddings reduce response latency
- Vector search provides fast semantic retrieval

**Security**

- User authentication and document-level access control
- Secure API communication and protected file storage

**Reliability**

- Retry handling for failed API calls
- Graceful handling of parsing and model-service failures

**Maintainability**

- Clear separation of frontend, backend, processing, and storage responsibilities
- Easy replacement of vector DB or LLM provider if needed

**Usability**

- Simple upload-and-chat workflow
- Minimal technical complexity for end users

## 2.6 API Catalogue

A possible high-level API catalogue is:

**Authentication APIs**

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`

- `POST /api/documents/upload`
- `GET /api/documents`
- `GET /api/documents/{documentId}`
- `DELETE /api/documents/{documentId}`

### Processing APIs

- `GET /api/documents/{documentId}/status`
- `POST /api/documents/{documentId}/process`

### Chat / Query APIs

- `POST /api/chat/query`
- `GET /api/chat/history/{documentId}`
- `DELETE /api/chat/history/{sessionId}`

### Admin / Monitoring APIs

- `GET /api/health`
- `GET /api/metrics`

---

# 3. Data Design

## 3.1 Data Model

The high-level data model includes the following entities:

### User

- user_id
- name
- email
- password_hash
- role
- created_at

- document_id
- user_id
- file_name
- file_type
- upload_time
- processing_status
- storage_path

**Chunk**

- chunk_id
- document_id
- chunk_text
- chunk_index
- page_number or source_reference

**Embedding**

- embedding_id
- chunk_id
- vector_data or vector_reference
- embedding_model
- created_at

**ChatSession**

- session_id
- user_id
- document_id
- started_at

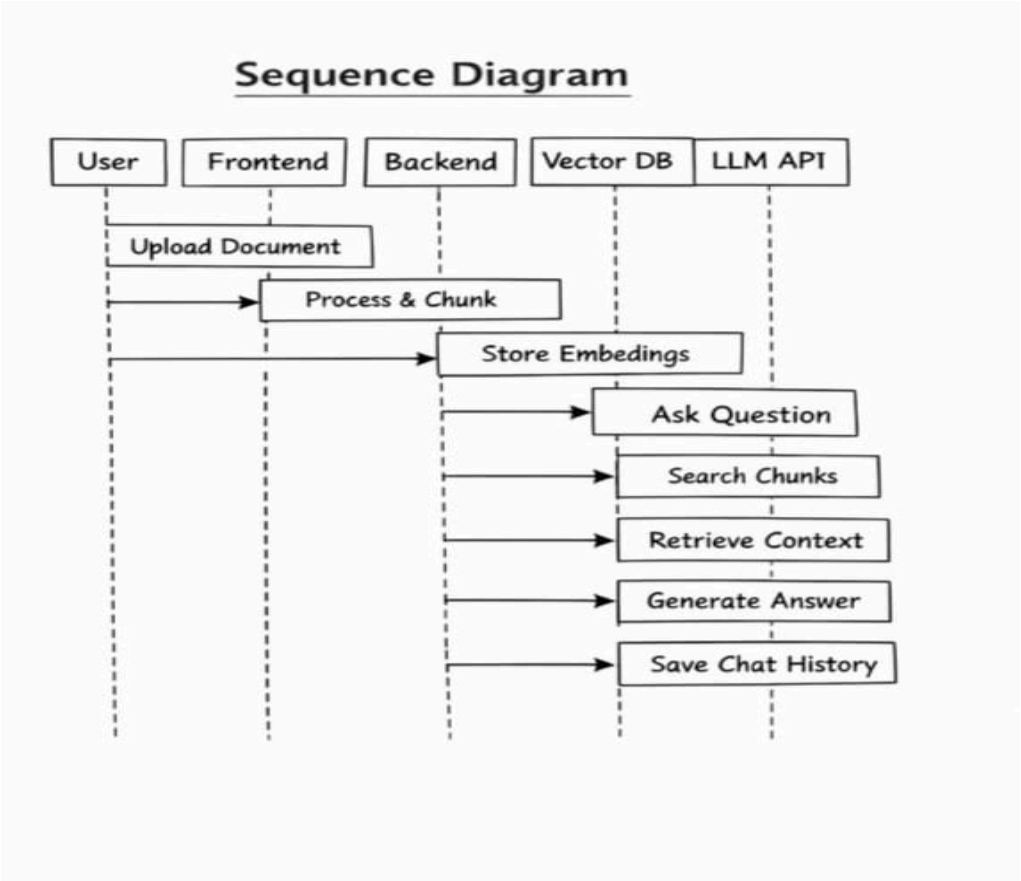
**ChatMessage**

- message_id
- session_id
- sender_type
- message_text
- timestamp

**GeneratedResponse**

- response_id
- session_id
- query_text
- retrieved_chunks
- response_text
- generated_at

This data model supports the key data elements from the LLD: document text, chunks, embeddings, queries, retrieved context, and responses.




## 3.2 Data Access Mechanism

- Relational DB access through ORM or repository layer

- Vector retrieval through vector DB SDK/API
- File access through secure file storage paths or object storage
- Backend service acts as the single controlled data access point
- No direct frontend access to databases

## 3.3 Data Retention Policies

Suggested policy:

- uploaded documents retained until user deletes them
- processed embeddings retained as long as the source document exists
- chat history retained for a defined period such as 30–90 days
- logs retained for troubleshooting and audit as per project policy
- deleted documents should also remove related chunks and embeddings

### 3.4 Data Migration

- initial migration creates user, document, session, and chat tables
- schema changes handled through migration tools
- vector store migration supported through re-embedding and re-indexing if embedding model changes
- metadata backup maintained before migration

---

## 4. Interfaces

The system interacts through the following interfaces:

### User Interface

- Web application for upload, document view, and chat

### Backend Interface

- REST APIs exposed over HTTPS

### Document Processing Interface

- Internal service interface for parsing and chunking

### Vector Database Interface

- Similarity search and vector insertion API

### LLM Interface

- Prompt-based request/response integration with external LLM service

### Authentication Interface

for secured access

- Login/token validation interface

---

## 5. State and Session Management

- User authentication state maintained using session token or JWT
- Each chat interaction linked to a session ID
- Frontend maintains current selected document and active chat context
- Backend persists chat history for continuity
- Document processing state includes statuses such as:
  - Uploaded
  - Processing
  - Ready
  - Failed

The LLD specifically mentions document readiness status and chat history storage, which supports this state model.

---

## 6. Caching

Caching can be introduced to improve responsiveness:

- cache document metadata for recently accessed documents
- cache recent chat history per session
- cache frequent vector search results for repeated queries
- cache authentication/session validation where appropriate
- avoid caching highly sensitive full-document text in insecure layers

A practical approach is to use in-memory cache or Redis for temporary query/session optimization.

---

# 7. Non-Functional Requirements

## 7.1 Security Aspects

Security is a key design consideration already highlighted in the LLD.

The system should include:

- authentication and authorization
- secure password storage using hashing
- HTTPS for all client-server communication
- secure document storage
- user-level isolation of documents and chats
- input validation for uploads and query requests
- API rate limiting
- logging and audit trail for important actions

## 7.2 Performance Aspects

Performance is also explicitly emphasized in the LLD through precomputed embeddings and fast vector retrieval.

The expected performance goals are:

- fast upload acknowledgment
- asynchronous document processing where needed
- low-latency semantic search
- optimized top-k retrieval before LLM invocation
- bounded prompt size to reduce response time
- scalable handling of multiple simultaneous user queries

---

# 8. References

The LLD cites the following core references, which are also appropriate for the HLD:

- OpenAI API Documentation
- Hugging Face Transformers Documentation
- FAISS / Pinecone / ChromaDB Documentation
- FastAPI / Flask Documentation
- Research papers on Retrieval-Augmented Generation (RAG)